

PRAKTIKUM SISTEM OPERASI

MODUL 11
(MEMORI XINU)



Oleh:

(Reyner Atira Prasetyo)

(2311104057)

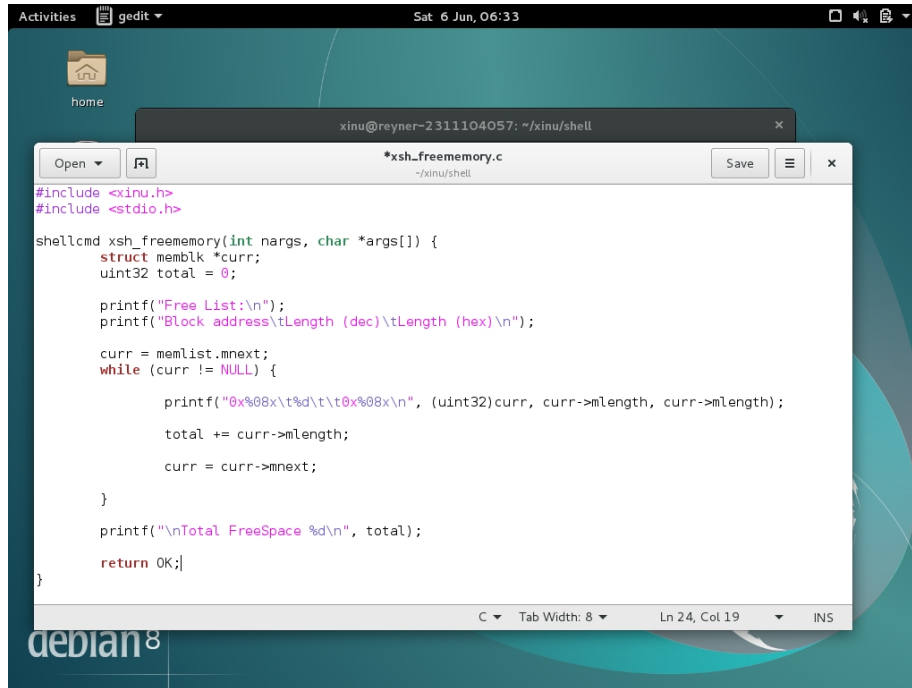
PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2026

JURNAL 11

1. Buatlah perintah baru bernama freememory

a. Buat xsh_freememory



The screenshot shows a gedit window titled "xsh_freememory.c" with the following code:

```
#include <xinu.h>
#include <stdio.h>

shellcmd xsh_freememory(int nargs, char *args[]) {
    struct memblk *curr;
    uint32 total = 0;

    printf("Free List:\n");
    printf("Block address\tLength (dec)\tLength (hex)\n");

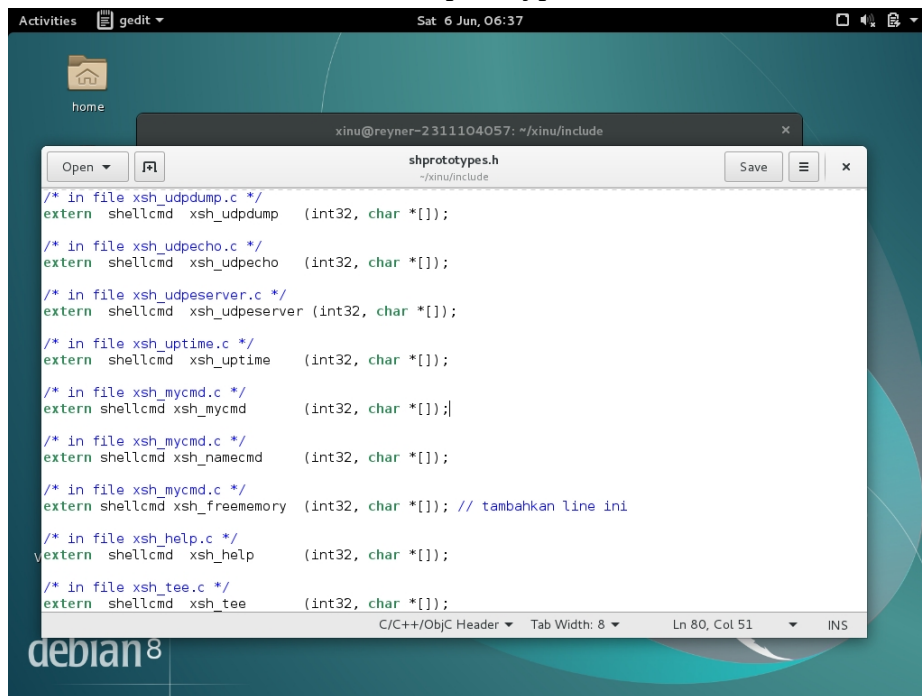
    curr = memlist.mnext;
    while (curr != NULL) {

        printf("0x%08x\t%d\t\t0x%08x\n", (uint32)curr, curr->mlength, curr->mlength);
        total += curr->mlength;
        curr = curr->mnext;
    }

    printf("\nTotal FreeSpace %d\n", total);

    return OK;
}
```

b. Daftarkan di file header shprototypes.h



The screenshot shows a gedit window titled "shprototypes.h" with the following code:

```
/* in file xsh_udpdump.c */
extern shellcmd xsh_udpdump (int32, char *[]);

/* in file xsh_udpecho.c */
extern shellcmd xsh_udpecho (int32, char *[]);

/* in file xsh_udpserver.c */
extern shellcmd xsh_udpserver (int32, char *[]);

/* in file xsh_uptime.c */
extern shellcmd xsh_uptime (int32, char *[]);

/* in file xsh_mycmd.c */
extern shellcmd xsh_mycmd (int32, char *[]);

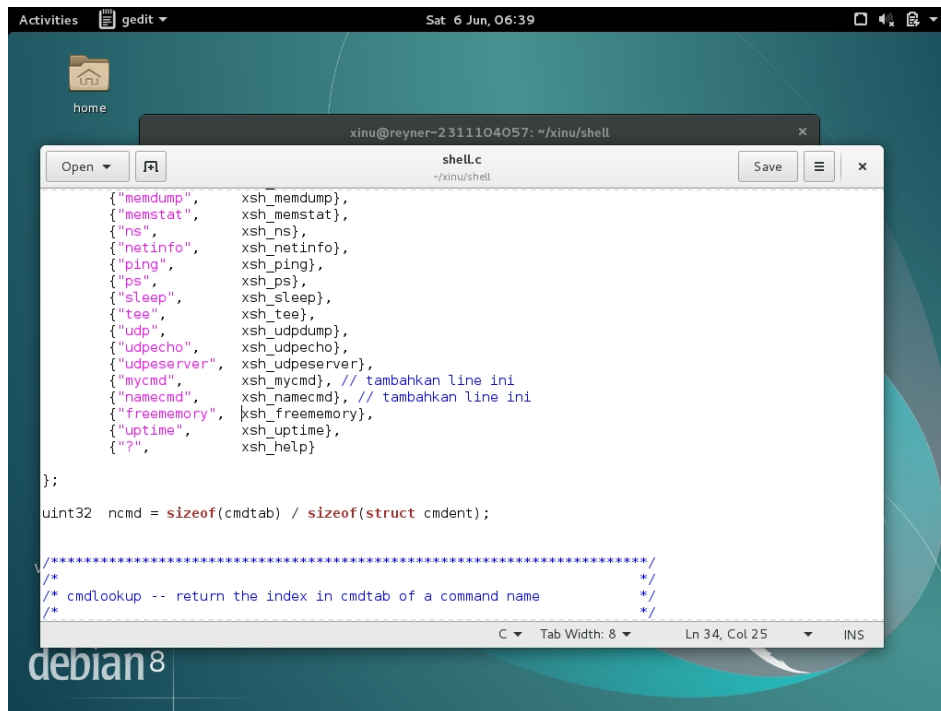
/* in file xsh_mycmd.c */
extern shellcmd xsh_namecmd (int32, char *[]);

/* in file xsh_mycmd.c */
extern shellcmd xsh_freememory (int32, char *[]); // tambahkan line ini

/* in file xsh_help.c */
extern shellcmd xsh_help (int32, char *[]);

/* in file xsh_tee.c */
extern shellcmd xsh_tee (int32, char *[]);
```

c. Daftarkan di file shell.c



```

xinu@reyner-2311104057: ~/xinu/shell
shell.c
~/xinu/shell

{"memdump",    xsh_memdump},
{"memstat",    xsh_memstat},
{"ns",         xsh_ns},
{"netinfo",    xsh_netinfo},
{"ping",       xsh_ping},
{"ps",         xsh_ps},
{"sleep",      xsh_sleep},
{"tee",        xsh_tee},
{"udp",        xsh_udpdump},
{"udpecho",    xsh_udpecho},
{"udpeserver", xsh_udpeserver},
{"mycmd",      xsh_mycmd}, // tambahkan line ini
{"namecmd",    xsh_namecmd}, // tambahkan line ini
{"freememory", xsh_freememory},
{"uptime",     xsh_uptime},
{"?",         xsh_help}

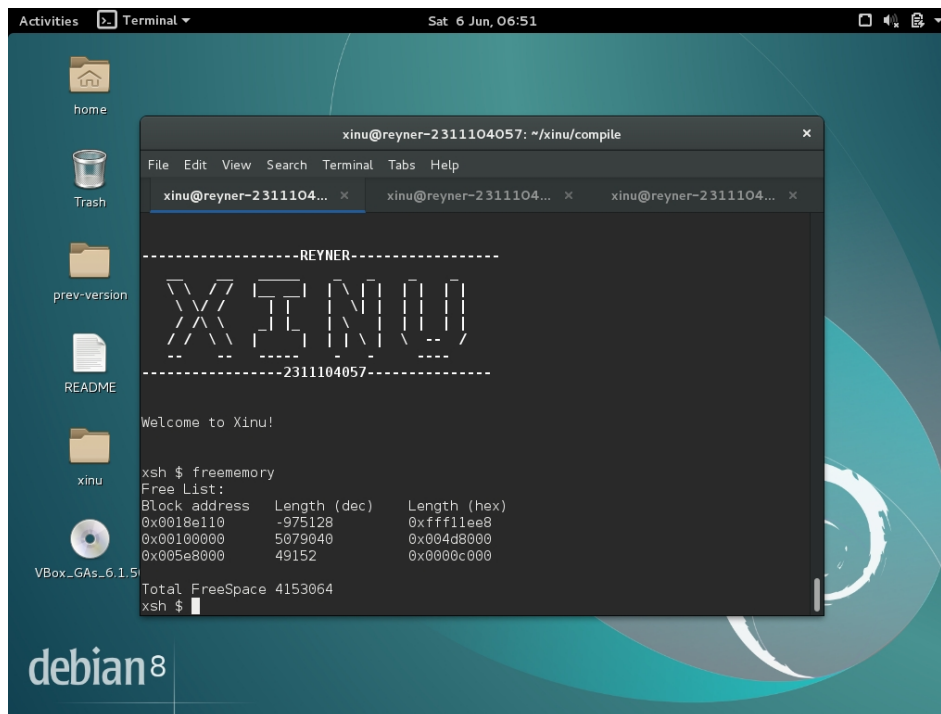
};

uint32 ncmd = sizeof(cmdtab) / sizeof(struct cmdent);

/*
 * cmdlookup -- return the index in cmdtab of a command name
 */

```

d. Hasil run



```

xinu@reyner-2311104057: ~/xinu/compile
File Edit View Search Terminal Tabs Help
xinu@reyner-2311104... xinu@reyner-2311104... xinu@reyner-2311104...

-----REYNER-----
XINU
-----2311104057-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
0x0018e110     -975128    0xffff11ee8
0x00100000     5079040    0x004d8000
0x005e8000      49152     0x0000c000

Total FreeSpace 4153064
xsh $

```

Kode untuk perintah freememory di atas bekerja dengan cara menelusuri linked list global pada sistem operasi Xinu yang bernama memlist. Menggunakan pointer curr, program akan melakukan iterasi pada setiap blok memori yang kosong untuk mencetak alamat blok (block address) beserta ukurannya (length) dalam format

desimal dan heksadesimal ke layar terminal. Selama proses penelusuran dari satu blok ke blok lainnya tersebut, program juga terus menjumlahkan ukuran masing-masing blok ke dalam variabel total sehingga dapat menghitung dan menampilkan keseluruhan kapasitas ruang memori yang masih kosong (total free space) di akhir eksekusi perintah.

2. Jawablah pertanyaan berikut:

- a. Mengapa Xinu memisahkan data segment dan BSS segment?

Sistem operasi memisahkan Data segment dan BSS segment untuk mengoptimalkan ukuran file executable dan efisiensi memori. Data segment digunakan untuk menyimpan variabel global atau statik yang sudah diinisialisasi (memiliki nilai awal), sehingga nilainya harus disimpan secara fisik di dalam file executable. Sebaliknya, BSS (Block Started by Symbol) segment menampung variabel global/statik yang belum diinisialisasi. Daripada menyimpan nilai nol (0) untuk variabel-variabel tersebut dan membuang ruang penyimpanan, OS hanya mencatat berapa besar ukuran blok memori yang dibutuhkan BSS. Ruang tersebut baru akan dialokasikan dan diisi dengan nol secara otomatis oleh OS saat program dimuat ke memori (runtime).

- b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Setiap kali fungsi alokasi (seperti `getmem()`) dipanggil, OS akan mencari blok memori yang cukup dari daftar free list dan memberikannya kepada proses, yang secara langsung akan mengurangi total free space. Sebaliknya, ketika proses memanggil fungsi dealokasi (seperti `freemem()`), blok memori tersebut dikembalikan ke dalam free list, sehingga menambah ukuran free space. Eksekusi yang dinamis ini menyebabkan ukuran free space terus berfluktuasi. Jika proses ini terjadi terus-menerus dengan blok memori yang ukurannya bervariasi, hal ini dapat menyebabkan memori terpecah-pecah (fragmentasi eksternal).

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap dikelola secara manual oleh programmer secara dinamis, sehingga sangat rentan terhadap human error. Programmer harus mengalokasikan dan mendealokasikan memori sendiri. Jika lupa mendealokasi, akan terjadi memory leak (kebocoran memori). Jika mengakses memori yang sudah didealokasi, akan terjadi masalah dangling pointer. Selain itu, heap rentan terhadap fragmentasi. Di sisi lain, stack sangat terstruktur (menggunakan prinsip LIFO - Last In First Out) dan

dikelola secara otomatis oleh compiler. Memori pada stack langsung dibebaskan begitu fungsi selesai dieksekusi, sehingga jauh lebih aman dari kebocoran memori.

- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Struktur linked list memungkinkan Xinu untuk mengelola daftar ruang memori kosong yang letaknya tidak saling berdekatan (non-contiguous) di memori fisik secara fleksibel dan dinamis. Setiap free block mengandung pointer (seperti mnext) yang menunjuk ke blok kosong berikutnya beserta informasi ukurannya (mlength). Hal ini memudahkan sistem operasi untuk menelusuri (traverse) memori saat mencari blok kosong yang ukurannya cukup untuk melayani permintaan alokasi memori yang dinamis, serta memudahkan proses penyisipan kembali (insertion) blok memori saat terjadi dealokasi.

- e. Apa tantangan utama dalam penggunaan heap di Xinu?

Tantangan paling utama adalah Fragmentasi Eksternal. Seiring berjalannya alokasi dan dealokasi, memori kosong di heap akan terpecah menjadi banyak blok-blok kecil yang saling terpisah. Walaupun secara total free space yang tersisa masih cukup besar, permintaan alokasi untuk memori berukuran besar bisa saja gagal dilayani karena tidak ada satu blok contiguous (berdekatan) yang ukurannya memenuhi spesifikasi permintaan tersebut. Tantangan lainnya adalah overhead performa karena CPU membutuhkan waktu komputasi ekstra untuk menelusuri linked list (free list) setiap kali harus mencari blok yang ukurannya cocok.